DS605 · Fundamentals of Machine Learning

# Lecture Notes 13–14

*Optimization: How Do We Actually Find a Good Model?*

> **NOTE · What you'll leave with**
>
> A working intuition for five hyperparameter-search strategies, and the ability to write gradient descent from scratch — by hand and in code. Two very different search problems hide inside every machine learning project, and this lecture untangles both of them from first principles, with real numbers and real code.

## Roadmap

- Part 1 — Choosing hyperparameters: manual search, grid search, random search, coarse-to-fine, and Bayesian optimization.
- Part 2 — Gradient descent: how a model learns its own parameters once the hyperparameters are fixed.

# Part 1 — Choosing Hyperparameters

## Every ML Method Has Three Ingredients

Pedro Domingos' well-known framework describes what any learning algorithm needs, in three parts:

- **Representation —** what shape can the model take? A tree, a line, a neural network.
- **Evaluation —** how do we score it? Accuracy, MSE, log-loss.
- **Optimization —** how do we search for a good one? This is what today is entirely about, and it turns out "optimization" secretly means two different things.

## Two Very Different Kinds of "Tuning"

Picture a Random Forest deciding whether a loan applicant will default. Before it ever sees data, you make some choices. After that, the algorithm makes its own.

| | Chosen by |
|---|---|
| Hyperparameters | You, beforehand — e.g., how many trees to grow, how deep each tree may go. You set these before training starts, and you have to search for good values. |
| Parameters | The algorithm, during training, e.g., the actual splits each tree makes. Once hyperparameters are fixed, the algorithm works these out on its own from the data. |

**KEY TAKEAWAY**

Both of these are search problems. Part 1 covers how to search for good hyperparameters. Part 2 is about how the algorithm searches for good parameters.

## Running Example: A Random Forest for Loan Default

Throughout Part 1, we tune a Random Forest that predicts loan default from an applicant's data. It has two hyperparameters we need to choose:

- $n_{\text{estimators}}$ — how many trees vote in the forest. More trees usually mean a steadier answer, but each tree costs time to train.
- max_depth — how many yes/no questions each tree may ask before deciding. Too shallow underfits; too deep overfits.

Every choice is scored the same honest way: train on the training set, measure validation accuracy, and never touch the test set until the very end.

## Search Method 1 of 5: Manual Search

The simplest possible strategy: pick a few combinations by hand, based on intuition or past experience, and see what happens.

| Attempt | $n_{\text{estimators}}$ | max_depth | Val. accuracy |
|---|---|---|---|
| 1 | 100 | 5 | 81.2% |
| 2 | 100 | 10 | 85.7% |
| 3 | 300 | 10 | 87.1% |

**KEY TAKEAWAY · Upside**

Fast to start, uses your intuition, and costs nothing to set up.

**COMMON MISTAKE · Downside**

Doesn't scale past a couple of hyperparameters, and it's easy to miss a better combination you never thought to try.

## Search Method 2 of 5: Grid Search

### Starting with just one knob

Before tuning two hyperparameters at once, tune only max_depth. Pick a short list of candidate values and try every single one:

| max_depth | Val. accuracy |
|---|---|
| 2 | 74.0% |
| 5 | 83.1% |
| 10 | 87.4% |
| 15 | 86.9% |
| 20 | 85.2% |
| 30 | 84.0% |

Nothing clever here — we simply tried every value on the list and kept the best one. The word “grid” only starts to matter once there's more than one hyperparameter.

### Adding a second knob

Do the same thing for n_estimators, and try every combination of the two lists. That combination grid is where the name comes from:

$n_{\text{estimators}} \in \{100, 200, 300, 400\}$

max_depth $\in \{5, 10, 15, 20, 25, 30\}$

**4 × 6 = 24 models to train.**

```
# scikit-learn does the looping for you
from sklearn.model_selection import GridSearchCV
grid = {'n_estimators': [100,200,300,400],
        'max_depth': [5,10,15,20,25,30]}
search = GridSearchCV(rf_model, grid, cv=5)
search.fit(X_train, y_train)
```

### The catch: the grid grows multiplicatively, not additively

Every extra hyperparameter multiplies the work; it doesn't just add to it:

| Hyperparameters tuned | Values each | Total combinations |
|---|---|---|
| 1 (max_depth) | 6 | 6 |
| 2 (+ n_estimators) | 6 × 4 | 24 |
| 3 (+ min_samples_leaf) | × 5 | 120 |
| 5 (+ max_features, bootstrap) | × 2 × 2 | 480 |

**COMMON MISTAKE**

480 configurations × 5-fold cross-validation = 2,400 total model fits — before you've tuned anything else. Every one of those fits is scored on validation folds, never on the test set; the test set stays locked until the final single check.

## Search Method 3 of 5: Random Search

Instead of every combination, draw a fixed number of random combinations from the same ranges and only try those:

```
from sklearn.model_selection import RandomizedSearchCV
dist = {'n_estimators': range(50,500),
        'max_depth': range(2,30)}
search = RandomizedSearchCV(rf_model, dist,
             n_iter=30,   # only try 30 combinations
             cv=5)
```

Surprising fact: with several hyperparameters, some matter far more than others. Grid Search wastes effort testing every value of the ones that don't matter. Random Search doesn't know which is which either — but by sampling combinations instead of a rigid grid, it naturally covers more distinct values of every hyperparameter for the same budget.

### Grid Search vs. Random Search

| | Grid Search | Random Search |
|---|---|---|
| Coverage per hyperparameter | Fixed, sparse | Broader, for the same budget |
| Continuous ranges | Must pick discrete points | Samples naturally |

| | Grid Search | Random Search |
|---|---|---|
| Cost | Grows multiplicatively | You choose the budget directly |
| Guarantees the best in the grid? | Yes, within the grid | No — only a good chance |

> **NOTE**
> Both still share the same weakness, though: neither one learns anything from the trials it has already run. Trial #12 is chosen exactly the same way, whether trial #11 was great or terrible.

## Search Method 4 of 5: Coarse-to-Fine

A simple trick that already starts to fix the "no memory" problem: search wide first, then search narrow around whatever looked best.

- **Pass 1 (wide net):** max_depth ∈ {2, 8, 14, 20, 26} — best around 14.
- **Pass 2 (zoom in):** max_depth ∈ {11, 13, 15, 17} — best around 15.

It works, but it's still you deciding where to zoom, by eye, one pass at a time. The next method automates exactly this instinct.

## Pause and Connect the Dots

- **Manual —** relies entirely on your intuition — doesn't scale.
- **Grid —** exhaustive, but the cost multiplies with every hyperparameter.
- **Random —** better coverage, but still picks trial #12 with zero memory of trial #11.
- **Coarse-to-fine —** learns a little — but only because you looked at the plot and zoomed in by hand.

> **CAUTION**
> The natural next question: what if the search itself could look at the plot and decide where to zoom — automatically, after every single trial?

## Search Method 5 of 5: Bayesian Optimization

The idea in one sentence: use every trial so far to make an educated guess about where to try next.

To see this clearly, zoom back down to a single hyperparameter — a learning rate η being tuned for a different model, somewhere between $10^{-5}$ and 1. (We switch examples for a moment purely because it's easier to see the idea on one number line; we return to the forest right after.)

| Trial | $\eta$ | Val. loss |
|---|---|---|
| 1 | 0.0001 | 0.75 |
| 2 | 0.1 | 0.63 |
| 3 | 0.01 | 0.34 |

Lower loss is better. Trial 3 is clearly the best of the three — a plain average of the three losses would throw that information away.

### After just three trials, a pattern emerges

A human looking at that table would think: “The region near 0.01 looks promising — let me try something close to it next,” rather than jumping to a completely unrelated value.

| Trial | $\eta$ | Val. loss | What we're thinking |
|---|---|---|---|
| 1 | 0.0001 | 0.75 | too small — barely learns |
| 2 | 0.1 | 0.63 | too big — also weak |
| 3 | 0.01 | 0.34 | promising middle ground |
| 4 | 0.006 | 0.29 | try nearby, not randomly |
| 5 | 0.004 | 0.27 | refine further |
| 6 | 0.0048 | 0.255 | converging |

**THINK ABOUT IT**

That instinct — “look near what worked” — is exactly what Bayesian Optimization automates. The rest of this section is just making that instinct precise.

### Ingredient 1 of 2: a cheap stand-in for the real thing

Training the actual model is expensive — it can take minutes or hours per trial. So we built a second, much cheaper model whose only job is to guess what the real one would say.

Think of it like a student who has taken three practice quizzes. After quiz 3, they don't just know their three scores — they have formed a rough mental picture of the whole subject: which topics they're confident about, and which they've barely touched and could be a pleasant (or unpleasant) surprise.

**NOTE**

This cheap stand-in is called the surrogate model. Given the trials run so far, it does two things across the search space: predicts how good a hyperparameter value is likely to be and reports its confidence in that prediction.

Fitted to the six η trials above, the surrogate produces a best-guess curve together with a shaded uncertainty band. The band pinches to almost nothing right at each point we've actually tried, and widens out between and beyond them. That pinch-and-widen pattern is the whole point: near a point we've actually tried, we're confident; far from any point we've tried, we're not — and the surrogate says so honestly, instead of pretending to know.

## The Gaussian Process, and two symbols we'll reuse

The most common surrogate model is called a Gaussian Process — a way of saying "a prediction, plus honest uncertainty about that prediction" in math:

$$f(x) \sim N(\mu(x), \sigma^2(x))$$

- $\mu(x)$ — the surrogate's best guess at hyperparameter value x (the gold line on the plot).
- $\sigma(x)$ — how uncertain that guess is (the width of the shaded band).

You won't need to derive this formula by hand — libraries handle the mathematics. What matters is reading μ and σ off a plot like the one just described.

## Ingredient 2 of 2: Where should we try next?

Having μ(x) and σ(x) everywhere doesn't answer this by itself. Two candidate points can both look tempting, for different reasons:

**WORKED EXAMPLE · Exploitation**

Try somewhere the surrogate already predicts will score well, like going back to a restaurant you know is good.

**THINK ABOUT IT · Exploration**

Try somewhere the surrogate is very uncertain — it might hide something even better, like trying a new restaurant you know nothing about.

A rule that balances both is called an acquisition function. It turns "good prediction" and "high uncertainty" into a single score for every candidate point, so we can just pick the highest one.

## Turning the balance into a formula

One simple, widely-used version, for a loss we want to minimize — the confidence bound:

$$a(x) = \mu(x) - \kappa \cdot \sigma(x)$$

- $\mu(x)$ — predicted loss — the surrogate's best guess at how good x is.
- $\sigma(x)$ — uncertainty — subtracting more of it rewards unexplored regions.

- $\kappa$ — a dial you set: how much to reward exploring the unknown.

**WORKED EXAMPLE · Small κ**

Mostly exploitation — trust the prediction.

**THINK ABOUT IT · Large κ**

Mostly exploration — chase the unknown.

We then simply pick the x with the lowest a(x) as the next trial. Dragging κ toward 0 snaps the recommendation to the best-known point; dragging it toward 3 sends it out into the least-explored region.

### One full iteration, back on our forest

Back to tuning n_estimators and max_depth. Every round of Bayesian Optimization is this six-step loop:

1. Fit the surrogate to every trial run so far.
2. Compute the acquisition score a(x) across the search space — cheap, milliseconds.
3. Pick the point that scores highest.
4. Actually train the forest with that setting — the expensive step.
5. Record the result.
6. Repeat until the budget runs out; return the best forest actually trained.

**KEY TAKEAWAY**

Notice the pattern in practice: the first click or two mostly explores empty regions; once a good neighborhood is found, later trials exploit it, refining ever closer to the optimum.

## All Five Search Methods, Side by Side

| | Manual | Grid | Random | Coarse-to-fine | Bayesian |
|---|---|---|---|---|---|
| Uses earlier trials? | Sort of | No | No | By hand | Automatically |
| Handles many hyperparameters? | Poorly | Poorly | OK | OK | Well |
| Continuous ranges? | Awkward | Must discretize | Natural | Natural | Natural |
| Setup effort | None | Low | Low | Medium | Higher |
| Best for | Quick baseline | Tiny, cheap spaces | Medium spaces | Manual refinement | Expensive models |

**NOTE**

Grid Search spends experiments. Random Search gambles them. Bayesian Search invests them — each trial is chosen because the last one taught it something.

## So, Which One Should You Actually Reach For?

- **1–2 hyperparameters, cheap model —** Grid Search; simple and exhaustive is fine.
- **Several hyperparameters, moderate cost —** Random Search; a good default almost everywhere.
- **Each training run takes minutes or hours —** Bayesian Optimization; worth the setup cost.
- **Quick sanity check, early in a project —** Manual Search, then Coarse-to-Fine once something works.

In practice: start with manual, then move to random for a broad sweep, and bring in Bayesian Optimization once training runs become expensive enough that each trial is precious.

## Before We Move On: Part 1 in One Breath

- Hyperparameters are choices you make before training — and finding good ones is itself a search problem.
- Grid and Random Search don't remember what they've already tried.
- Bayesian Optimization fixes that with two ingredients: a surrogate model that predicts performance and its own uncertainty, and an acquisition function that balances exploiting what looks good against exploring what's unknown.

> **CAUTION**
> One thing we deliberately skipped: once n_estimators and max_depth are fixed, how does the forest itself learn? That's a completely different kind of search — and it's next.

# Part 2 — Gradient Descent

We've picked hyperparameters from the outside. Now: how does a model find the actual parameter values — weights, splits, coefficients — that make it fit the data?

> **NOTE · New running example**
> We switch to Linear Regression — predicting an exam score from the number of hours studied. It has a smooth loss surface, which makes the idea of gradient descent easy to actually see.

## Predicting an Exam Score From Hours Studied

Five students. We want a straight line through this data:

| Hours studied (x) | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| Exam score (y) | 52 | 58 | 65 | 72 | 80 |

$$\hat{y} = \theta_0 + \theta_1 x$$

- $\theta_0$ — the intercept — predicted score with zero hours studied.
- $\theta_1$ — the slope — extra points per extra hour studied.

Training means: find the values of $\theta_0$ and $\theta_1$ that fit this data best. That's it — that's the whole problem.

## How Do We Know If $\theta_0$, $\theta_1$ Are Good?

Same idea as Part 1: score every guess honestly, with a number. Try $\theta_0$=40, $\theta_1$=5 and see what it predicts:

| x | y | predicted 40+5x | error² |
|---|---|---|---|
| 1 | 52 | 45 | 49 |
| 2 | 58 | 50 | 64 |
| 3 | 65 | 55 | 100 |
| 4 | 72 | 60 | 144 |
| 5 | 80 | 65 | 225 |

$$L = 1/5 \cdot (49+64+100+144+225) = 116.4$$

This is the Mean Squared Error — average the squared gaps between prediction and truth. Lower is better; 0 would mean a perfect fit.

## So, How Do We Find the Best $\theta_0$, $\theta_1$?

$$\theta_0^*, \theta_1^* = \text{argmin}_{\theta 0, \theta 1}\, L(\theta_0, \theta_1)$$

This should look familiar — it's the exact same shape of problem as Part 1! We could try a grid of $\theta_0$, $\theta_1$ values and keep the best one. That would work here... but it breaks down fast: a real neural network can have millions of parameters, and no grid could ever cover that space.

> **NOTE**
> We need a method that doesn't blindly guess — one that looks at the loss right where it stands and knows immediately which direction to move. That method is Gradient Descent.

### Why not just use calculus and solve it directly?

For this one simple case — a straight line, one input — you actually can solve for the exact best $\theta_0$, $\theta_1$ with a formula (it's called the normal equation, derived in full in the next lecture's notes).

> **KEY TAKEAWAY · Works here**
> Small, simple models with a closed-form solution.

> **COMMON MISTAKE · Breaks down fast**
> Logistic regression, neural networks, almost everything else — no formula exists.

> **CAUTION**
> Gradient Descent is the one technique that works for both cases, which is exactly why it's worth learning properly here, on the simple case, before you meet it again in a model with no shortcut available.

## The Big Idea, Before Any Formula: Walking Downhill in Thick Fog

You're standing somewhere on the loss surface. You can't see the bottom — the fog is too thick. But you can feel which way the ground slopes under your feet.

Formally: at every point, we compute the gradient — the direction of steepest increase — and step in the opposite direction, since we want to go down.

### What does "feel the slope" mean in math?

Forget the exam example for one moment. Take the simplest bowl-shaped loss imaginable, with a single number w:

$$L(w) = w^2$$

The derivative dL/dw tells us the slope at any point w. For $L(w)=w^2$, that's a rule from calculus:

$$\mathrm{dL/dw} = 2w$$

> **WORKED EXAMPLE · At w=3**
> slope = 6 — positive, so downhill is to the left.

**WORKED EXAMPLE · At w=−3**

slope = −6 — negative, so downhill is to the right.

That's the whole trick: the sign of the slope always points away from downhill, so we always step in the opposite direction of the slope.

### Now there are two slopes, not one

Our loss depends on two numbers, $\theta_0$ and $\theta_1$ — so we need the slope in each direction separately. Together they're called the gradient:

$$\partial L/\partial\theta_0 = (2/n)\,\Sigma_i\,(\hat{y}_i - y_i)$$

$$\partial L/\partial\theta_1 = (2/n)\,\Sigma_i\,(\hat{y}_i - y_i)\cdot x_i$$

In words, both are just the average prediction error, one plain, and one weighted by how many hours the student studied. Neither formula needs to be memorized — libraries compute this for you — but knowing it's "just the average error" demystifies the whole thing.

## The Gradient Descent Update Rule

One line, applied to every parameter, over and over:

$$\theta \leftarrow \theta - \eta \cdot \partial L/\partial\theta$$

- $\theta$ — whichever parameter we're updating — $\theta_0$ or $\theta_1$.
- $\partial L/\partial\theta$ — the slope at the current value — which way is uphill.
- $\eta$ — the learning rate — a small number you choose, controlling step size.

In plain English: new value = old value, nudged a little in the downhill direction. That's the entire algorithm — the rest of this section is just consequences of this one line.

### Watching it actually run, step by step

Starting from $\theta_0 = \theta_1 = 0$, learning rate $\eta = 0.01$, on our five students:

| Step | θ0 | θ1 | Loss |
|---|---|---|---|
| 0 | 0.00 | 0.00 | 4375.4 |
| 1 | 1.31 | 4.20 | 2666.1 |
| 2 | 2.34 | 7.40 | 1669.2 |
| 3 | 3.15 | 9.84 | 1087.4 |
| 4 | 3.81 | 11.69 | 747.7 |
| … after 2,000 steps | 44.4 | 7.0 | 0.24 |

Notice: the loss drops every single step — 4375 → 2666 → 1669 → 1087 → 748 — even while $\theta_1$ temporarily overshoots past its final resting value. That's normal. After many more steps, it settles near $\theta_0 \approx 44.4$, $\theta_1 \approx 7.0$ — matching the exact closed-form answer from the previous section.

### Gradient descent from scratch, in NumPy

Every line here maps directly onto something we just did by hand:

```
import numpy as np

x = np.array([1,2,3,4,5])
y = np.array([52,58,65,72,80])
n = len(x)

theta0, theta1 = 0.0, 0.0
eta = 0.01

for step in range(2000):
    pred = theta0 + theta1 * x
    error = pred - y                    # same as the table's error column
    grad0 = (2/n) * np.sum(error)
    grad1 = (2/n) * np.sum(error * x)
    theta0 = theta0 - eta * grad0       # the update rule, applied twice
    theta1 = theta1 - eta * grad1

print(theta0, theta1)   # ≈ 44.4  7.0
```

Real libraries (PyTorch, TensorFlow, scikit-learn's SGDRegressor) do exactly this loop under the hood — just with faster, more careful implementations of the same four lines.

## Choosing η Badly Breaks Everything

**WORKED EXAMPLE · Too small**

Many tiny steps — correct direction, painfully slow.

**CAUTION · Just right**

Converges in a handful of steps.

**COMMON MISTAKE · Too large**

Overshoots the minimum, may bounce or diverge entirely.

## A Subtler Way η Can Misbehave: Feature Scale

Imagine a second feature measured in the thousands (like household income) alongside hours studied (1–5). One gradient will be huge, the other tiny — no single η suits both.

- **Unscaled features —** a long, narrow valley; the path zig-zags slowly toward the bottom.
- **Scaled features —** a round bowl; a straight, quick path to the bottom.

The fix: subtract the mean and divide by the standard deviation for every feature first — the same idea as z-scores from statistics.

## How Much Data Per Step?

Our worked example used all five students to compute the gradient for each student. That's a choice — and not the only one:

**WORKED EXAMPLE · Batch — all 5 students, every step**

Smooth, accurate gradient. Slow if the dataset is huge.

**THINK ABOUT IT · Stochastic — 1 student, every step**

Fast, noisy updates — five updates per pass through the data.

**CAUTION · Mini-batch — e.g., 2 students, every step**

The practical middle ground almost everyone actually uses.

The update rule above doesn't change at all — only how many rows are used to compute the average error each time.

### Epoch vs. batch vs. iteration

Say we use mini-batches of 2, on our 5 students:

- **Batch size —** how many examples per update — is 2 here.
- **Iteration —** one single update — one pass of the training loop.
- **Epoch —** one full pass through all the data — here, that's $\lceil 5/2 \rceil = 3$ iterations.

**NOTE**

Training “for 10 epochs” with a batch size of 2 on our 5 students means 30 total iterations — 3 per epoch, repeated 10 times.

## η Doesn't Have to Stay Fixed

Take bigger steps early, when you're far from the minimum and speed matters. Take smaller steps late, when precision matters more:

```
eta = eta0 / (1 + decay * step)   # simple decay schedule
```

This is exactly like the coarse-to-fine idea from Part 1, but it runs automatically at every step rather than by hand between passes.

## Not Every Loss Surface Is a Clean Bowl

Linear regression's loss is convex — one bowl, one bottom. Neural networks are not so kind. In practice, this matters less than it sounds — mini-batch noise, momentum, and modern optimizers (Adam, RMSprop) mostly avoid getting permanently stuck.

## A Practical Gradient Descent Playbook

1. Scale your features first.
2. Start with a small-ish learning rate; watch the loss curve, not just the final number.
3. Loss bouncing wildly or growing? $\eta$ is too big — cut it.
4. Loss barely moving after many steps? $\eta$ is too small, or you've converged — check both.
5. Use mini-batches for anything beyond toy-sized data.

## Two Nested Optimization Problems, Working Together

**NOTE · Outer loop · Part 1 — Choose hyperparameters**

Manual, Grid, Random, Coarse-to-Fine, or Bayesian Search — chosen by you, before training.

**NOTE · Inner loop · Part 2 — Learn the parameters**

Gradient Descent — learned automatically by the algorithm, during training.

**KEY TAKEAWAY**

Every time you fit a model, both loops are running: an outer search over settings, and an inner search over weights. This lecture was built on first principles.

## Closing: We Can Now Score a Hypothesis, and Search for a Good One

- **Representation —** done.
- **Evaluation —** done.
- **Optimization —** done.

**NOTE**

Next: put all three pieces together to train the first complete model — Linear Regression.